实　　验　　报　　告

学　　号	2108001 2078	姓　　名	姚嫣如	专业班级	网络空间安全 23
课程名称	**网络空间安全导论**			学期	**2025 秋**
任课教师	曲海鹏、李晓慧		完成日期		2025/10/28
小组成员	姚嫣如				

一、实验内容及步骤（55%）

（一）使用 C 语言编写以下程序并按要求完成实验，实验每一步均需要截图：使用递归的方法计算 1+2+…+n（n<=1000）的和，若输入的 n 不是正整数则返回 0，若 n 超出取值范围返回-1。

1. 将该程序保存，文件名为 lab4.c
2. 使用 gcc 编译该程序（两种方法），最终的可执行文件名为 lab，要求生成所有警告信息
3. 仿照课上的例子定义宏，练习使用-D 参数
4. 删除刚才定义的宏，重新编译，并开启 PIE，强度为 1
5. 练习使用 gdb 的常用命令调试该程序，需要练习的命令至少包括运行、退出、查看本程序代码、设置断点、删除断点、设置断点状态、打印变量、单步调试、栈操作、显示断点、显示寄存器信息、显示栈帧、显示参数和局部变量、打印栈帧、以某种格式打印某一地址处开始的 N 个长度单元为 U 的内存值。

注：具体操作自定但要包括需要练习到的内容，对于一些命令，尽可能解释各字段含义，如 info frame 命令。

（二）自行了解 Makefile 相关内容，并编写 Makefile 文件，并写一个 Makefile 的 QuickStart，附上截图和文字说明。

二、实验结果（35%）

（一）使用 C 语言编写以下程序并按要求完成实验：

使用递归的方法计算 1+2+…+n（n<=1000）的和，若输入的 n 不是正整数则返回 0，若 n 超出取值范围返回-1。

如下为该递归代码：

```c
#include<stdio.h>
#include<stdlib.h>

//定义取值范围
#ifdef MAX_N_MACRO
#define MAX_LIMIT MAX_N_MACRO
#else
#define MAX_LIMIT 1000 //默认取值范围 n <= 1000
#endif

//累加函数
int Sum(int n)
{
        if (n == 1)
            return n;
        else
            return Sum(n-1) + n;
}

int main()
{
        int n, ans;
        scanf("%d", &n);

        if (n <= 0)
        {
            printf("0\n");
            return 0;
        }
        else if (n > MAX_LIMIT)
        {
                printf("-1\n");
                return 0;
        }

        ans = Sum(n);

        printf("%d", ans);
        return 0;
}
```

1.将该程序保存，文件名为 lab4.c



先通过 touch 新建一个文件，命名为 lab4.c，将代码内容通过 vim 复制粘贴到该文件中，手动处理格式问题，并保存退出。

```
        return n;
    else
        return Sum(n-1) + n;
}

int main()
{
    int n, ans;
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("0\n");
        return 0;
    }
    else if (n > MAX_LIMIT)
    {
        printf("-1\n");
        return 0;
    }

    ans = Sum(n);
    printf("%d", ans);
    return 0;
}

lab4.c [+]
-- INSERT --
```

```
stu112@19b39220d0e1:~$ touch lab4.c
stu112@19b39220d0e1:~$ ls
copy_of_test.txt  lab3  lab4.c  la.txt  test.txt  vimlab  word
stu112@19b39220d0e1:~$ vim lab4.c
stu112@19b39220d0e1:~$ cat lab4.c
#include<stdio.h>
#include<stdlib.h>

#ifdef MAX_N_MACRO
#define MAX_LIMIT MAX_N_MACRO
#else
#define MAX_LIMIT 1000
#endif

int Sum(int n)
{
    if (n == 1)
        return n;
    else
        return Sum(n-1) + n;
}

int main()
{
    int n, ans;
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("0\n");
        return 0;
    }
    else if (n > MAX_LIMIT)
    {
        printf("-1\n");
        return 0;
    }

    ans = Sum(n);
    printf("%d", ans);
    return 0;
}
```

2. 使用 gcc 编译该程序（两种方法），最终的可执行文件名为 lab，要求生成所有警告信息

（1）方法一：预处理 -> 编译 -> 汇编 -> 链接

预处理：gcc -E -Wall lab4.c -o lab.i

对头文件和宏进行了预处理，但源文件不改变。

```
stu112@19b39220d0e1:~$ gcc -E -Wall lab4.c -o lab.i
stu112@19b39220d0e1:~$ cat lab.i
# 1 "lab4.c"
# 1 "<built-in>"
# 1 "<command-line>"
# 31 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 32 "<command-line>" 2
# 1 "lab4.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 27 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 461 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 1 3 4
# 452 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 453 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/long-double.h" 1 3 4
# 454 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 462 "/usr/include/features.h" 2 3 4
# 485 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 1 3 4
# 10 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs-64.h" 1 3 4
# 11 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 2 3 4
# 486 "/usr/include/features.h" 2 3 4
# 34 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 2 3 4
# 28 "/usr/include/stdio.h" 2 3 4
# 1 "/usr/lib/gcc/x86_64-linux-gnu/9/include/stddef.h" 1 3 4
# 209 "/usr/lib/gcc/x86_64-linux-gnu/9/include/stddef.h" 3 4
```

```
# 10 "lab4.c"
int Sum(int n)
{
    if (n == 1)
        return n;
    else
        return Sum(n-1) + n;
}

int main()
{
    int n, ans;
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("0\n");
        return 0;
    }
    else if (n > 1000)
    {
        printf("-1\n");
        return 0;
    }

    ans = Sum(n);
    printf("%d", ans);
    return 0;
}
```

编译：gcc -S -Wall lab4.c -o lab.s

或者 gcc -S -Wall lab.i -o lab.s

将代码文件转成编译文件，文件中可看到代码所包含的两个函数：Sum 和 main

```
stu112@19b39220d0e1:~$ gcc -S -Wall lab4.c -o lab.s
stu112@19b39220d0e1:~$ cat lab.s
        .file   "lab4.c"
        .text
        .globl  Sum
        .type   Sum, @function
Sum:
.LFB6:
        .cfi_startproc
        endbr64
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        subq    $16, %rsp
        movl    %edi, -4(%rbp)
        cmpl    $1, -4(%rbp)
        jne     .L2
        movl    -4(%rbp), %eax
        jmp     .L3
.L2:
        movl    -4(%rbp), %eax
        subl    $1, %eax
        movl    %eax, %edi
        call    Sum
        movl    -4(%rbp), %edx
        addl    %edx, %eax
.L3:
        leave
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc
.LFE6:
        .size   Sum, .-Sum
        .section        .rodata
.LC0:
        .string "%d"
.LC1:
        .string "0"
.LC2:
        .string "-1"
```

```
        .type   main, @function
main:
.LFB7:
        .cfi_startproc
        endbr64
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        subq    $16, %rsp
        movq    %fs:40, %rax
        movq    %rax, -8(%rbp)
        xorl    %eax, %eax
        leaq    -16(%rbp), %rax
        movq    %rax, %rsi
        leaq    .LC0(%rip), %rdi
        movl    $0, %eax
        call    __isoc99_scanf@PLT
        movl    -16(%rbp), %eax
        testl   %eax, %eax
        jg      .L5
        leaq    .LC1(%rip), %rdi
        call    puts@PLT
        movl    $0, %eax
        jmp     .L8
.L5:
        movl    -16(%rbp), %eax
        cmpl    $1000, %eax
        jle     .L7
        leaq    .LC2(%rip), %rdi
        call    puts@PLT
        movl    $0, %eax
        jmp     .L8
.L7:
        movl    -16(%rbp), %eax
        movl    %eax, %edi
        call    Sum
        movl    %eax, -12(%rbp)
        movl    -12(%rbp), %eax
        movl    %eax, %esi
        leaq    .LC0(%rip), %rdi
        movl    $0, %eax
```

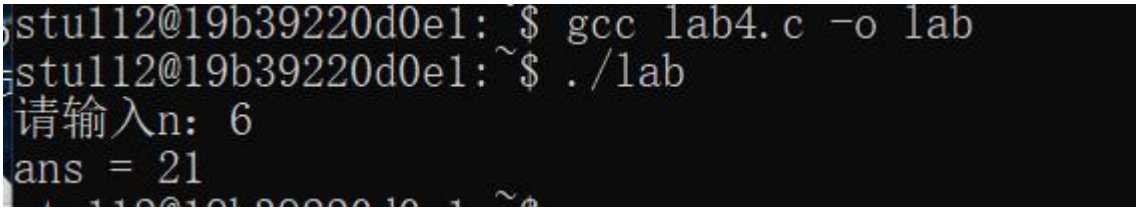
汇编：gcc -c -Wall lab.s -o lab.o

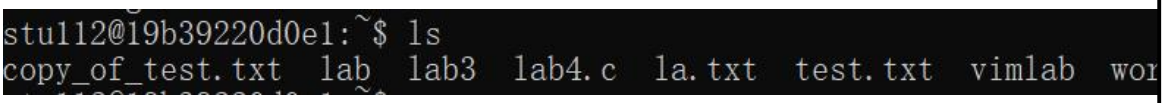
```
stu112@19b39220d0e1:~$ gcc -c lab.s -o lab.o
stu112@19b39220d0e1:~$ cat lab.o
□□□HH□}□□□
```

（2）方法二：直接生成 gcc lab4.c -o lab

（已通过 vim 修改了输出内容）

此时并不会生成.i/.s/.o 相关文件，会直接一部生成可执行文件。



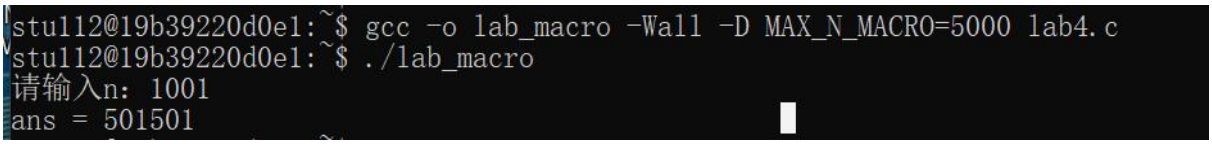


3. 仿照课上的例子定义宏，练习使用-D 参数

gcc -o lab_macro -Wall -D MAX_N_MACRO=5000 lab4.c

生成一个lab_macro的宏文件,并生成所有的警告信息,声明代码中定义的宏值为5000。

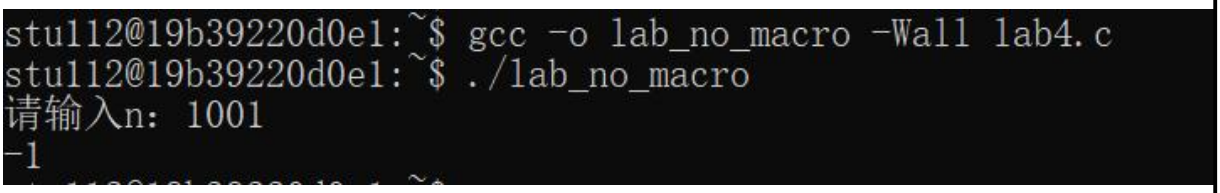
此时，如果输入 n=1001（超过 MAX_LIMIT），仍然可得到计算结果，代表宏值定义成功。



4. 删除刚才定义的宏，重新编译，并开启 PIE，强度为 1

（1）gcc -o lab_no_macro -Wall lab4.c

删除宏定义后，重新输入 1001，会发现由于越界，此时输出-1。



（2）gcc -o lab_pie -Wall -fPIE -pie -O1 lab4.c

-fPIE：（编译阶段）生成位置无关的可执行代码，程序加载到内存时，其指令不依赖于固定的内存地址，而是通过相对地址访问数据，可防止 “缓冲区溢出” 等攻击利用固定地址跳转。

-pie：（链接阶段）将编译后的代码链接为位置无关可执行文件，与 -fPIE 配合使用，最终生成的 lab_pie 是一个支持地址随机化（ASLR）的可执行程序，增强程序的安全性。

```
stu112@19b39220d0e1:~$ gcc -o lab_pie -Wall -fPIE -pie -O1 lab4.c
lab4.c: In function ‘main’:
lab4.c:22:6: warning: ignoring return value of ‘scanf’, declared with attribute warn_unused_result [-Wunused-re
   22 |      scanf("%d", &n);
```

可以看到生成了一个动态链接，生成了一个随机 ID

```
stu112@19b39220d0e1:~$ file lab_pie
lab_pie: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.
so.2, BuildID[sha1]=3932ce835c994d27339363c883fe6086cfb6d204, for GNU/Linux 3.2.0, not stripped
```

5. 练习使用 gdb 的常用命令调试该程序，需要练习的命令至少包括运行、退出、查看本程序代码、设置断点、删除断点、设置断点状态、打印变量、单步调试、栈操作、显示断点、显示寄存器信息、显示栈帧、显示参数和局部变量、打印栈帧、以某种格式打印某一地址处开始的 N 个长度单元为 U 的内存值。

注：具体操作自定但要包括需要练习到的内容，对于一些命令，尽可能解释各字段含义，如 info frame 命令。

（1）使用-g 参数在目标文件中加入调试信息

gcc -g lab4.c -o lab

```
stu112@19b39220d0e1:~$ gcc -g lab4.c -o lab
```

（2）使用 list 显示全部代码，每次显示 10 行，后续可直接 l 或者回车，gdb 可自动化记忆上一次的命令。

list 1

```
pwndbg> list 1
1       #include<stdio.h>
2       #include<stdlib.h>
3
4       #ifdef MAX_N_MACRO
5       #define MAX_LIMIT MAX_N_MACRO
6       #else
7       #define MAX_LIMIT 1000
8       #endif
9
10      int Sum(int n)
pwndbg> l
11      {
12          if (n == 1)
13              return n;
14          else
15              return Sum(n-1) + n;
16      }
17
18      int main()
19      {
20          int n, ans;
pwndbg>
21          printf("请输入n: ");
22          if (scanf("%d", &n) != 1)
23          {
24              fprintf(stderr,"输入错误：请输入一个整数！\n");
25              return 1;
26          }
27
28          if (n <= 0)
29          {
30              printf("0\n");
```

```
25              return 1;
26          }
27
28          if (n <= 0)
29          {
30              printf("0\n");
pwndbg>
31              return 0;
32          }
33          else if (n > MAX_LIMIT)
34          {
35              printf("-1\n");
36              return 0;
37          }
38
39          ans = Sum(n);
40          printf("ans = %d\n", ans);
pwndbg>
41          return 0;
42      }
43
pwndbg>
```

（3）添加断点，并查看断点信息

添加：b 10

查看断点信息：info b

```
pwndbg> b 10
Breakpoint 1 at 0x11c9: file lab4.c, line 11.
pwndbg> info b
Num     Type           Disp Enb Address            What
1       breakpoint     keep y   0x00000000000011c9 in Sum at lab4.c:11
```

通过函数名添加断点：b Sum

新添加的断点编号会延续之前的编号继续往下，尽管删除了，也不会重新编号。

```
pwndbg> b Sum
Breakpoint 2 at 0x11c9: file lab4.c, line 11.
pwndbg> info b
Num     Type           Disp Enb Address            What
2       breakpoint     keep y   0x00000000000011c9 in Sum at lab4.c:11
```

（4）删除断点

删除：d 1 //删除编号为 1 的断点

```
pwndbg> info b
Num     Type           Disp Enb Address            What
1       breakpoint     keep y   0x00000000000011c9 in Sum at lab4.c:11
pwndbg> d 1
pwndbg> info b
No breakpoints or watchpoints.
```

（5）设置断点状态

无效：dis 3

将 3 号断点状态设置为无效，可以看到在 info 之后，3 号断点的 Enb 字段的值变为‘n’.

```
pwndbg> dis 3
pwndbg> info b
Num     Type           Disp Enb Address            What
2       breakpoint     keep y   0x00000000000011c9 in Sum at lab4.c:11
3       breakpoint     keep n   0x0000000000001267 in main at lab4.c:28
```

有效：ena 3

将 3 号断点状态设置为有效，可以看到在 info 之后，3 号断点的 Enb 字段的值变为‘y’.

```
pwndbg> ena 3
pwndbg> info b
Num     Type           Disp Enb Address            What
2       breakpoint     keep y   0x00000000000011c9 in Sum at lab4.c:11
3       breakpoint     keep y   0x0000000000001267 in main at lab4.c:28
```

（6）运行+调试

运行：run

运行之后可以看到代码停在断点第 28 行

```
[ SOURCE (CODE) ]
In file: /home/stu112/lab4.c
   23      {
   24          fprintf(stderr,"输入错误：请输入一个整数！\n");
   25          return 1;
   26      }
   27
 □28      if (n <= 0)
   29      {
   30          printf("0\n");
   31          return 0;
   32      }
   33      else if (n > MAX_LIMIT)
```

逐语句调试：s

代码进行 if 判断之后，到达下一步 else if 判断。

```
[ SOURC
In file: /home/stu112/lab4.c
   28      if (n <= 0)
   29      {
   30          printf("0\n");
   31          return 0;
   32      }
 □33      else if (n > MAX_LIMIT)
   34      {
   35          printf("-1\n");
   36          return 0;
   37      }
   38
```

逐过程调试：n

进入函数 Sum 后使用 next 调试，发现每次会实行一个完整的函数内容，通过递归重新回到函数开头。

```
In file: /home/stu112/lab4.c
    7 #define MAX_LIMIT 1000
    8 #endif
    9
   10 int Sum(int n)
   11 {
 □12      if (n == 1)
   13          return n;
   14      else
   15          return Sum(n-1) + n;
   16 }
   17
```

（7）查看函数压栈过程

查看栈帧：bt

```
pwndbg> bt
#0  Sum (n=10) at lab4.c:12
#1  0x0000562020b902a8 in main () at lab4.c:39
#2  0x00007f2e1dc15083 in __libc_start_main (main=0x562020b901f7 <main>, argc=1, argv=0x7fffc06a4b88, init=<optimized ou
t>, fini=<optimized out>, rtld_fini=<optimized out>, stack_end=0x7fffc06a4b78) at ../csu/libc-start.c:308
#3  0x0000562020b9010e in _start ()
```

（8）打印变量+查看参数信息、寄存器

在最后一次递归时，查看变量，通过 print 可以打印变量 n，当前值为 1.

在 bt 中可以看到每一次压栈过程。

```
pwndbg> print n
$9 = 1
pwndbg> info locals
No locals.
pwndbg> info args
n = 1
pwndbg> bt
#0  Sum (n=1) at lab4.c:16
#1  0x0000562020b901f0 in Sum (n=2) at lab4.c:15
#2  0x0000562020b901f0 in Sum (n=3) at lab4.c:15
#3  0x0000562020b901f0 in Sum (n=4) at lab4.c:15
#4  0x0000562020b901f0 in Sum (n=5) at lab4.c:15
#5  0x0000562020b901f0 in Sum (n=6) at lab4.c:15
#6  0x0000562020b901f0 in Sum (n=7) at lab4.c:15
#7  0x0000562020b901f0 in Sum (n=8) at lab4.c:15
#8  0x0000562020b901f0 in Sum (n=9) at lab4.c:15
#9  0x0000562020b901f0 in Sum (n=10) at lab4.c:15
#10 0x0000562020b902a8 in main () at lab4.c:39
#11 0x00007f2e1dc15083 in __libc_start_main (main=0x562020b901f7 <main>, argc=1, argv=0x7fffc06a4b88, init=<optimized ou
t>, fini=<optimized out>, rtld_fini=<optimized out>, stack_end=0x7fffc06a4b78) at ../csu/libc-start.c:308
#12 0x0000562020b9010e in _start ()
pwndbg>
```

info r 可以查看寄存器信息。

```
pwndbg> info r
rax            0x1                 1
rbx            0x562020b902e0      94695987938016
rcx            0x0                 0
rdx            0x0                 0
rsi            0x0                 0
rdi            0x1                 1
rbp            0x7fffc06a4950      0x7fffc06a4950
rsp            0x7fffc06a4940      0x7fffc06a4940
r8             0xa                 10
r9             0x0                 0
r10            0x7f2e1dd8cac0      139836045970112
r11            0x0                 0
r12            0x562020b900e0      94695987937504
r13            0x7fffc06a4b80      140736421579648
r14            0x0                 0
r15            0x0                 0
rip            0x562020b901f5      0x562020b901f5 <Sum+44>
eflags         0x246               [ PF ZF IF ]
cs             0x33                51
ss             0x2b                43
ds             0x0                 0
es             0x0                 0
fs             0x0                 0
gs             0x0                 0
k0             0x0                 0
k1             0x0                 0
k2             0x0                 0
k3             0x0                 0
k4             0x0                 0
k5             0x0                 0
k6             0x0                 0
k7             0x0                 0
```

（9）以某种格式打印某一地址处开始的 N 个长度单元为 U 的内存值。

x/5gx $rsp :16 进制展示 5 个长度单位内存值。

```
pwndbg> x/5gx $rsp
0x7fffc06a4940: 0x0000000000000000      0x0000000100000000
0x7fffc06a4950: 0x00007fffc06a4970      0x0000562020b901f0
0x7fffc06a4960: 0x0000034000000340
```

（二）自行了解 Makefile 相关内容，并编写 Makefile 文件，并写一个 Makefile 的 QuickStart，附上截图和文字说明。

（1）makefile

用于管理项目构建过程的工具，支持多个程序一起运行，是 make 命令所读取的配置文件，包含构建项目的规则，主要作用是检查项目文件的依赖关系，自动执行必要的命令，从而更新目标文件。主要包含一下三部分：

- **目标（Target）**：需要生成的文件，例如可执行文件。
- **依赖（Dependencies）**：生成目标所依赖的文件或目标。
- **命令（Commands）**：构建目标时需要执行的命令。

语法：

```
target : dependencies
    command
```

target:目标文件，可以是一个目标文件或一个动作名称

dependencies：生成目标所依赖的文件或其他目标

command：构建目标的命令，必须以 Tab 键开头。

（2）使用 makefile 一键执行

使用 vim 编写一个 makefile 文件，包含所有编译过程内容，并建立规则 build，一次性执行所有生成指令。

```
#预处理
lab4.i: lab4.c
    gcc -E lab4.c -o lab4.i
    @echo "预处理完成，生成lab4.i"
#编译
lab4.s: lab4.i
    gcc -S lab4.i -o lab4.s
    @echo "编译完成，生成lab4.s"
#汇编
lab4.o: lab4.s
    gcc -c lab4.s -o lab4.o
    @echo "汇编完成，生成lab4.o"
#链接
lab4: lab4.o
    gcc lab4.o -o lab4
    @echo "链接完成，生成可执行文件lab"
#清除所有生成文件
.PHONY: clean
clean:
    rm -f lab4.i lab4.s lab4.o
    @echo "清除完成"
#便捷目标：一次性执行所有命令
build: lab4.i lab4.s lab4.o lab4
    @echo "完整编译过程完成！"
```

```
stu112@19b39220d0e1:~$ make build
gcc -E lab4.c -o lab4.i
预处理完成，生成lab4.i
gcc -S lab4.i -o lab4.s
编译完成，生成lab4.s
gcc -c lab4.s -o lab4.o
汇编完成，生成lab4.o
gcc lab4.o -o lab4
链接完成，生成可执行文件lab
完整编译过程完成！
```

```
stu112@19b39220d0e1:~$ ls
copy_of_test.txt  lab3  lab4.c  lab4.o  lab_macro     lab_pie  makefile  vimlab
lab               lab4  lab4.i  lab4.s  lab_no_macro  la.txt   test.txt  word
```

使用 clean 指令，会清除所有中间文件

make clean

```
stu112@19b39220d0e1:~$ make clean
rm -f lab4.i lab4.s lab4.o
清除完成
stu112@19b39220d0e1:~$ ls
copy_of_test.txt  lab  lab3  lab4  lab4.c  lab_macro  lab_no_macro  lab_pie  la.txt  makefile  test.txt  vimlab  wc
```

三、心得总结（写出在完成实验过程中遇到的问题、解决方法，以及体会、收获等）（10%）

1、使用-Wall 时，提出了一个警告信息

```
stu112@19b39220d0e1:~$ gcc -o lab_pie -Wall -fPIE -pie -O1 lab4.c
lab4.c: In function ‘main’:
lab4.c:22:6: warning: ignoring return value of ‘scanf’, declared with attribute warn_unused_result [-Wunused-re
   22 |      scanf("%d", &n);
```

若用户输入的不是整数（如字母、符号），scanf 会读取失败，但代码会继续执行，可能使用未初始化的 n 导致逻辑错误；若输入流异常（如文件结束），程序也无法感知并处理。

通过 vim 修改代码如下：

```
    printf("请输入n: ");
    if (scanf("%d", &n) != 1)
    {
        fprintf(stderr,"输入错误：请输入一个整数！\n");
        return 1;
    }
```

2、心得体会

通过本次实验，了解 gcc 的编译程序的完整过程，以及相应的 Linux 指令，并学习了如何使用 gdb 在命令面板调试代码，以及添加断点、修改断点状态等。此外还自学了 makefile 的工作原理和操作过程，为将来进行实际大型项目时提前了解一些常用操作。